UMT

Database Systems

Unit 2 – Database Management System (DBMS)

School of Systems & Technology - SST

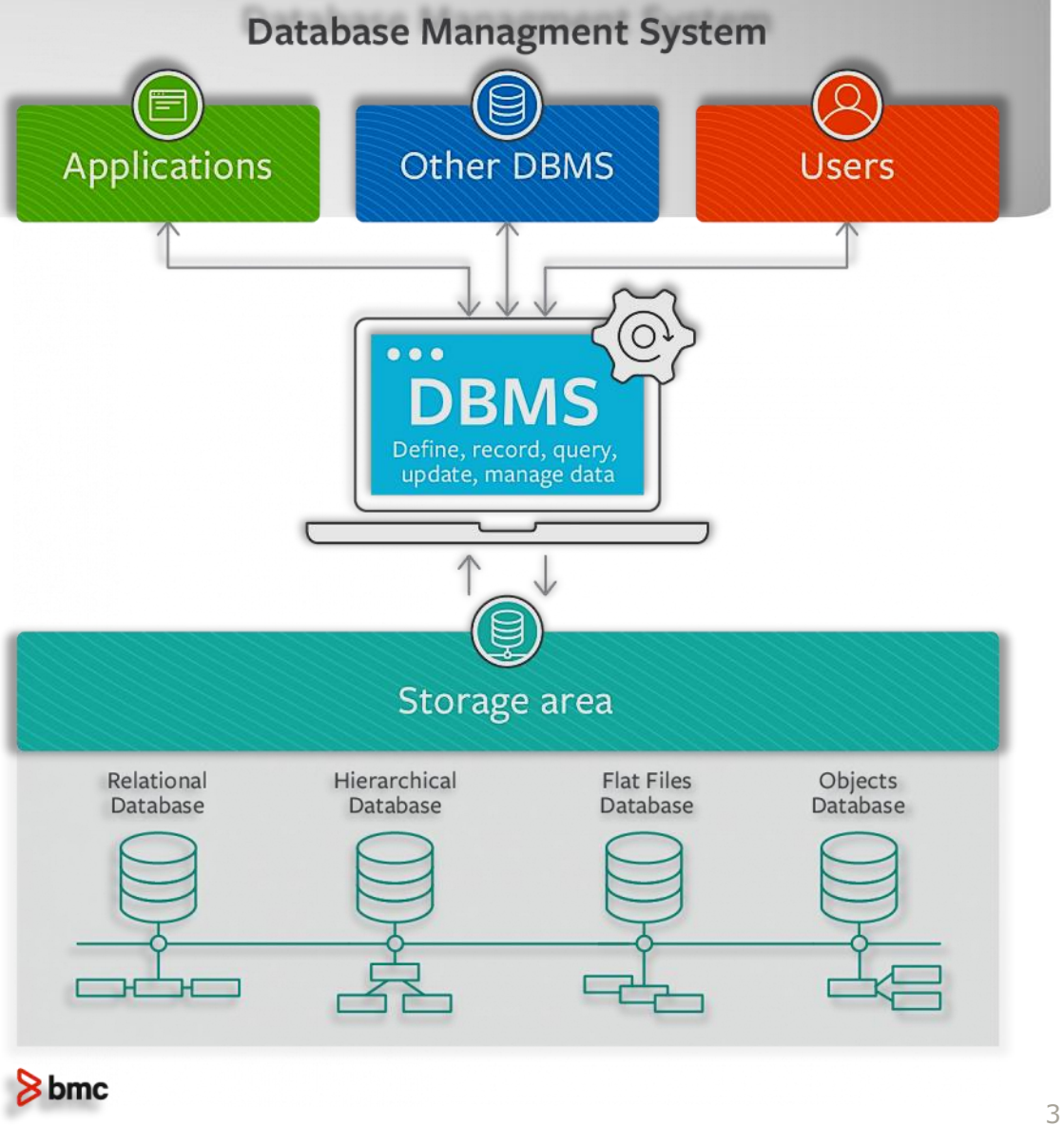
Mr. Maaz Waseem

What we will learn

- What is a Database Management System (DBMS)?
- Typical functions of a DBMS
- Major components of DBMS environment
- The database life cycle (DLC)
- Roles in the database environment
- History of databases / DBMS
- Advantages of DBMS
- Disadvantages of DBMS

What is a Database Management System?

- A database management system (DBMS) is system software for creating and managing databases.
- A DBMS makes it possible for end users to create, protect, read, update and delete data in a database.
- DBMS serves as an interface between databases and users or application programs, ensuring that data is consistently organized and remains easily accessible.
- Examples of DBMS include MySQL, Oracle Database, Microsoft SQL Server, and PostgreSQL.



Typical functions of DBMS

1. **Data Definition**: This involves defining the structure of the database, including creating, modifying, and deleting schemas, tables, views, indexes, etc.

2. **Data Manipulation**: DBMS allows users to interact with the data stored in the database. Common operations include inserting, updating, deleting, and querying data.

3. **Data Retrieval**: It enables users to retrieve specific information from the database using queries. Queries can range from simple to complex, using SQL or other query languages supported by the DBMS.

4. **Data Security**: DBMS provides mechanisms to ensure the security and integrity of the data. This includes user authentication, access control, encryption, and auditing.

5. **Data Integrity**: DBMS enforces data integrity constraints to maintain the accuracy and consistency of data. This includes primary key constraints, foreign key constraints, unique constraints, etc.

Typical functions of DBMS

6. **Concurrency Control**: DBMS manages concurrent access to the database by multiple users or applications to ensure data consistency and integrity.

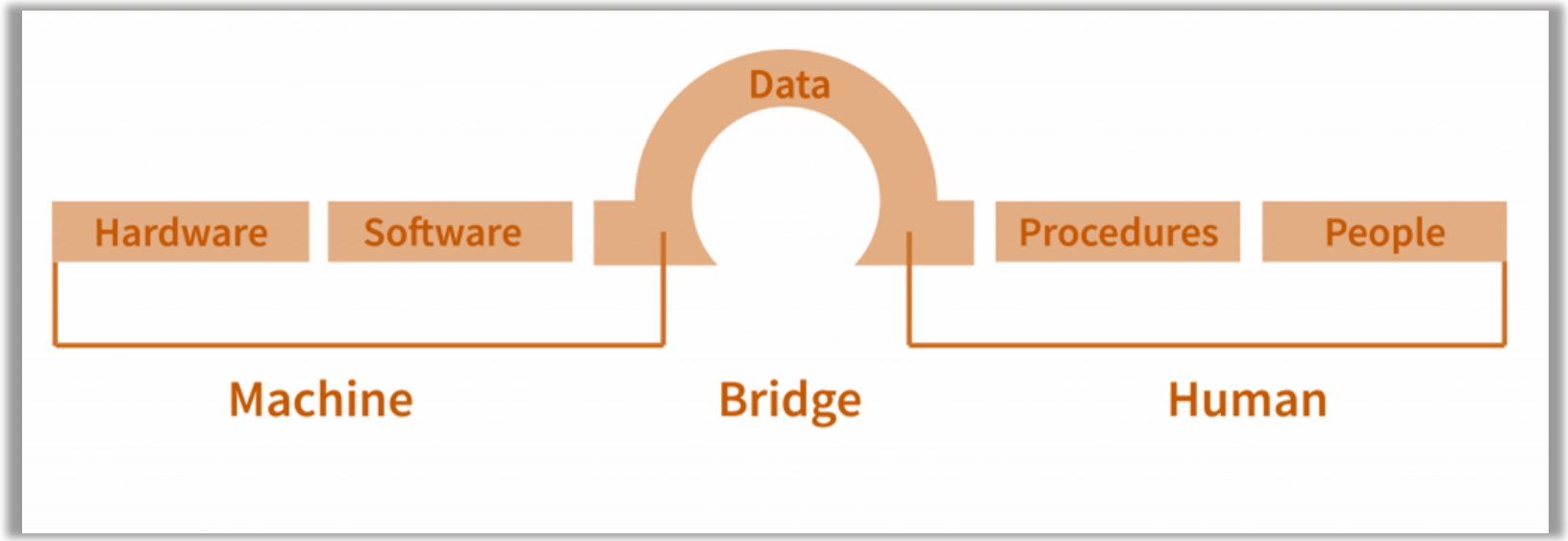
7. **Backup and Recovery**: DBMS provides mechanisms for backing up data periodically and recovering data in case of system failures, crashes, or other disasters.

8. **Transaction Management**: DBMS ensures the atomicity, consistency, isolation, and durability (ACID properties) of transactions. It supports transaction management through features like commit, rollback, save points, and transaction logging.

9. **Performance Tuning**: DBMS includes tools and techniques for optimizing the performance of database operations, such as query optimization, indexing, caching, and partitioning.

10. **Replication and Distribution**: Some DBMSs support replication and distribution of data across multiple servers or locations for scalability, fault tolerance, and improved performance.

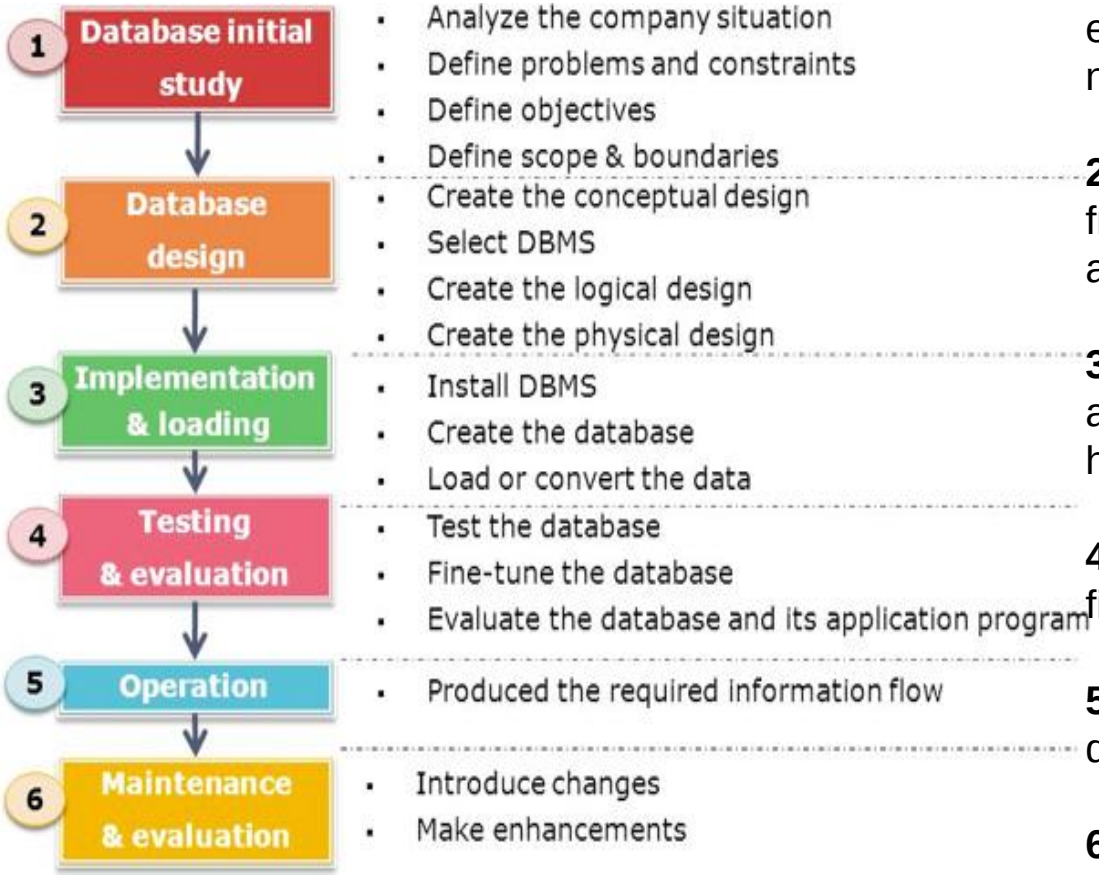
Major components of DBMS environment



A DBMS is a bridge between machines and humans. The Machine side (Hardware & Software) provides the tools, while the Human side (People & Procedures) provides the users and the rules. Data sits in the middle as the Bridge because it is the only thing both sides share: the machines store it, and the humans use it. Without Data, the technology has no purpose and the people have no information.

The Database Life Cycle (DBLC)

The **Database Life Cycle (DBLC)** is the step-by-step roadmap that transforms a business idea into a working digital library, guiding it from initial planning and design to daily use and long-term updates.



1. Database Initial Study: We analyze the business to figure out exactly what problems we are trying to solve and what the data needs to accomplish.

2. Database Design: We create the architectural blueprints, moving from a conceptual 'big picture' to the specific technical map of tables and relationships.

3. Implementation & Loading: We install the software, build the actual database structure, and pour the existing data into its new home.

4. Testing & Evaluation: We stress-test the system to find bugs and fine-tune performance before real users start relying on it.

5. Operation: The system goes live, and the database begins its daily job of providing the information the company needs to function.

6. Maintenance & Evaluation: We monitor the system's health and update it as the business grows or needs change over time.

Roles in the Database Environment

- A database system is usually used by many people in an organization.
- Each person has a different responsibility in managing, developing, and using the database.
- Instead of one person doing everything, different **roles** exist to make the system efficient, secure, and reliable.

Examples of responsibilities include:

- Managing the database
- Designing the database structure
- Developing applications
- Analyzing data

Roles in the Database Environment

The common roles involved in a database environment are:

1. Database Administrator (DBA)
2. Database Developer
3. Database Architect / Database Designer
4. Data Analyst
5. Application Developer
6. Data Steward
7. End Users

Each role focuses on a different aspect of working with data and databases.

Roles in the Database Environment
Database Administrator (DBA)

The **Database Administrator (DBA)** is responsible for managing and maintaining the database system.

Main responsibilities:

- Installing and configuring the database system
- Managing database security and user permissions
- Creating backups and recovering lost data
- Monitoring database performance
- Ensuring the database is always available and reliable

Example:

If the database crashes or data is lost, the DBA is responsible for fixing the issue and restoring the data.

Roles in the Database Environment
Database Developer

A **Database Developer** writes the code that interacts with the database. They focus on creating and maintaining database logic.

Main responsibilities:

- Writing SQL queries
- Creating stored procedures and triggers
- Optimizing queries for better performance

Example:

Writing a query that retrieves all students who enrolled this semester.

Roles in the Database Environment
Database Architect / Database Designer

The **Database Architect or Designer** plans how the database will be structured. They design the overall blueprint of the database before it is implemented.

Main responsibilities:

- Designing database schemas
- Defining tables, attributes, and relationships
- Creating ER diagrams
- Ensuring the design supports business requirements

Example:

Designing tables such as Students, Courses, and Enrollments, and defining how they relate to each other.

Roles in the Database Environment
Data Analyst

A **Data Analyst** studies data to discover useful information.
They use the database to support **decision making**.

Main responsibilities:

- Querying data from the database
- Creating reports and dashboards
- Identifying patterns and trends
- Helping organizations make data-driven decisions

Example:

Analyzing sales data to determine which product sells the most.

Roles in the Database Environment
Application Developer

An **Application Developer** builds software applications that use the database. These applications allow users to interact with the database easily.

Main responsibilities:

- Developing applications that connect to databases
- Implementing features such as login systems or data entry forms
- Sending queries to the database through the application
- Displaying results to users

Example:

Developing a university portal where students can register for courses.

Roles in the Database Environment
Data Steward

A **Data Steward** is responsible for ensuring the quality and integrity of data. They focus on maintaining accurate and consistent data across the organization.

Main responsibilities:

- Defining data standards
- Ensuring data accuracy and consistency
- Monitoring data quality
- Managing data policies and governance

Example:

Making sure student IDs follow a consistent format across all systems.

Roles in the Database Environment
End Users

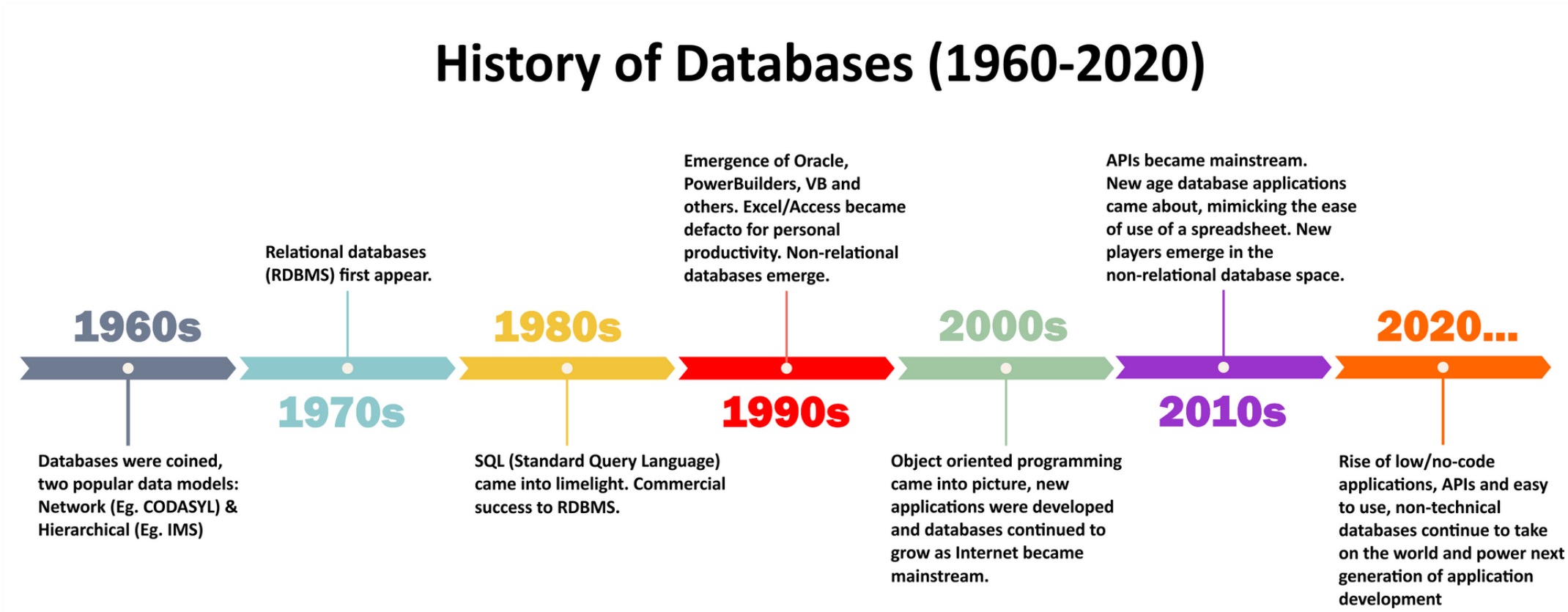
End Users are the people who use the database through applications. They usually do not interact with the database directly.

Examples of end users:

- Students using a university portal
- Bank customers using online banking
- Employees entering company data
- Managers viewing reports

End users rely on the database system to perform their daily tasks.

History of Databases / DBMS



Advantages of DBMS

1. **Data Integration:** Merges data from different sources into a single, unified database.
2. **Data Access:** Uses standard languages like SQL to retrieve information quickly and easily.
3. **Decision Making:** Delivers high-quality, organized data to help management make informed choices.
4. **Backup & Recovery:** Automates the process of saving data and restoring it after a system failure.
5. **Data Sharing:** Allows multiple departments or applications to use the same set of data.
6. **Data Integrity:** Enforces validation rules so only accurate, "clean" data enters the system.
7. **Data Security:** Protects sensitive information from unauthorized access through encryption and permissions.
8. **Data Privacy:** Ensures users only see the specific data permitted by their access level.
9. **Data Concurrency:** Allows many users to access and update the database at the same time without errors.
10. **Data Redundancy (Reduction):** Minimizes duplicate records to save storage space and reduce confusion.
11. **Data Inconsistency (Prevention):** Ensures that a change made in one place is reflected everywhere instantly.
12. **Multi-Access Support:** Interfaces with various platforms, including web, mobile, and desktop apps.

Disadvantages of DBMS

1. **Increased Cost:** High expenses for buying the software, powerful hardware, and licensing.
2. **Complexity:** The system is very intricate and difficult for most people to design or manage.
3. **Need of Skilled Staff:** Requires hiring high-paid experts (DBAs) to keep everything running properly.
4. **Performance:** It can be slower than simple files because the system has to follow so many rules.
5. **Frequency Update/Replacement Cycle:** Forced to buy new versions or better hardware quite often.
6. **Database Failure:** If the central system crashes, every single department stops working at once.
7. **Increased Vulnerability:** Putting all your data in one "basket" makes it a bigger target for hackers.
8. **Lower Efficiency:** For very small or simple tasks, a DBMS is often "overkill" and slows things down.

Thank you

Any Queries?